Course Project Requirements

1 Project Overview

The primary objective of this project is to bridge theory and practice by implementing two distinct data compression algorithms entirely from scratch. Teams will apply their custom algorithms to various datasets and perform a comprehensive mathematical and technical comparison based on the principles of Information Theory.

2 Team Formation and Rules

- Team Size: From 5 to 8 students per team (Registration Form).
- Programming Languages: Any programming language is allowed (e.g., C++, Python, Java, C#, MATLAB).
- You must implement the core compression and coding algorithms entirely from scratch. The use of built-in compression libraries or ready-made functions for the actual compression logic is strictly prohibited.

3 Core Project Requirements

Every team must complete the following core tasks:

- Algorithm Implementation: Select and implement two different lossless data compression algorithms from scratch (e.g., Huffman Coding, LZW, Arithmetic Coding, or Shannon-Fano).
- Graphical User Interface (GUI): Develop a user-friendly GUI for your application. The interface must allow users to select files, choose which compression/decompression algorithm to apply, and clearly display real-time statistics (such as original file size, compressed size, compression ratio, and execution time).
- Dataset Testing: Run both algorithms on at least three different types of files (e.g., a text document, a bitmap image, and a highly repetitive data file).
- Comparative Analysis: Compare the two algorithms against each other. You must calculate and evaluate:
  - The mathematical Entropy $H(X)$ of the original files.
  - The Compression Ratio achieved by each algorithm.
  - Execution time (compression and decompression speed).
  - Memory usage and efficiency of the data structures used.
- BONUS (Channel Noise & Error Correction): Teams that want to achieve the highest marks can attempt the bonus objective. In the real world, compressed data are highly vulnerable to corruption. To earn the bonus, your team must:
  - Simulate a Noisy Channel: After compressing the data, pass it through a simulated noisy environment (e.g., a Binary Symmetric Channel) that flips bits randomly based on a specific probability.

  - Implement Error Correction: Implement an error-correcting code (such as Hamming Codes or Reed-Solomon) from scratch to encode the compressed data before transmission and decode/correct the flipped bits upon reception, before finally decompressing the file.

4 Core Deliverables

Your final submission must include:

- Project Proposal: A one-page abstract defining the two algorithms chosen, the datasets you plan to use, whether you are attempting the bonus, and a clear breakdown of responsibilities for all 5–8 team members.
- Code: Well-documented code with a file explaining how to compile and run the application. Your code will be rigorously checked for library bypasses.
- Final Technical Report:
  - The Math: Explicitly define the Entropy $H(X)$ of your datasets.
  - The Comparison: Present your findings comparing the two algorithms using graphs, tables, and theoretical justifications for why one outperformed the other on specific file types.
- Presentation: A live demonstration of your GUI application compressing and decompressing files. Every member of the team must participate and be prepared to answer technical questions regarding their specific contributions.